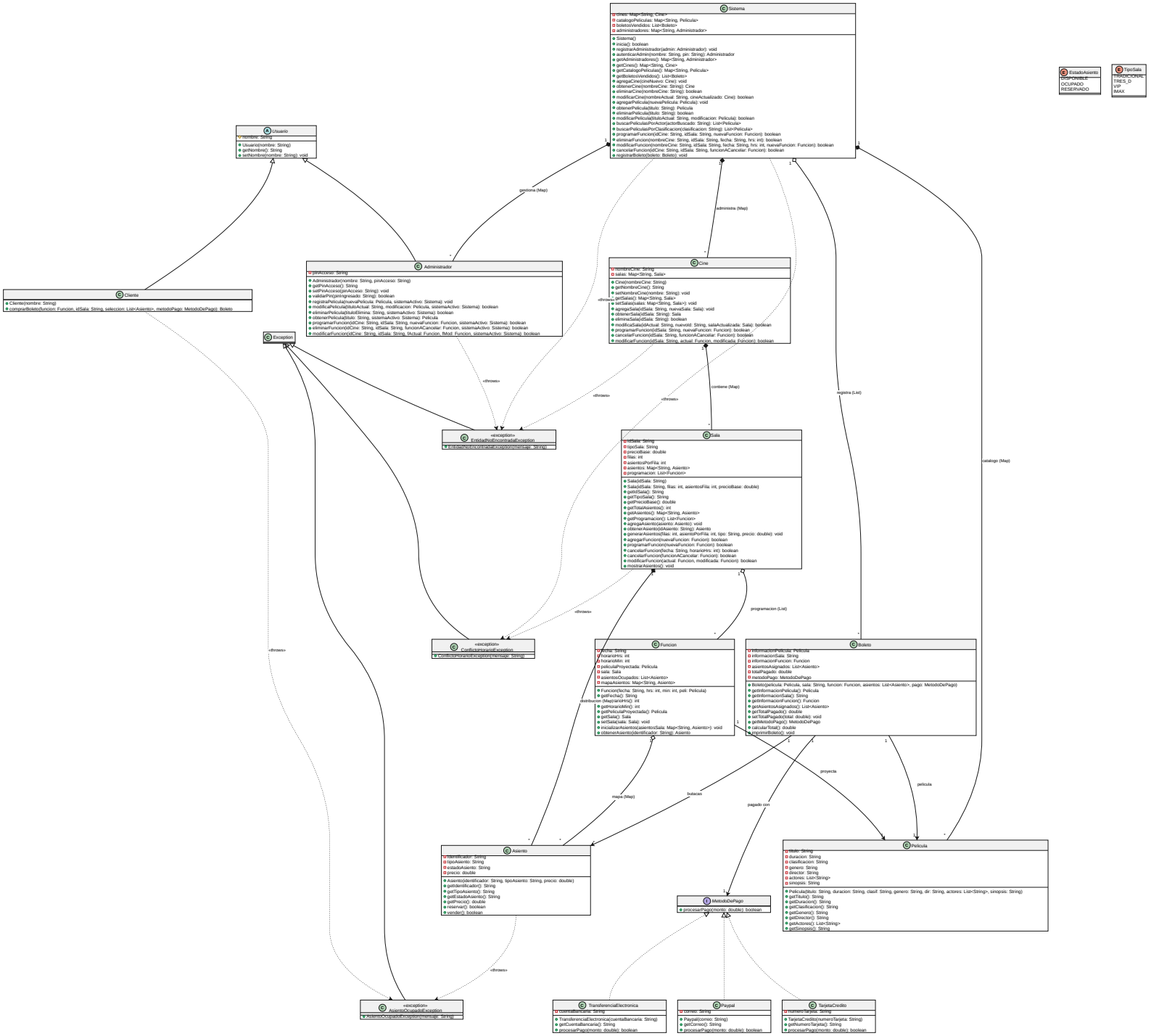

Sistema
-cines: Map<String, Cine>
-catalogoPeliculas: Map<String, Pelicula>
-boletosVendidos: List<Boleto>
-administradores: Map<String, Administrador>
+Sistema()
+inicia(): boolean
+registrarAdministrador(admin: Administrador): void
+autenticarAdmin(nombre: String, pin: String): Administrador
+getAdministradores(): Map<String, Administrador>
+getCines(): Map<String, Cine>
+getCatalogoPeliculas(): Map<String, Pelicula>
+getBoletosVendidos(): List<Boleto>
+agregarCine(cineNuevo: Cine): void
+obtenerCine(nombreCine: String): Cine
+eliminarCine(nombreCine: String): boolean
+modificarCine(nombreActual: String, cineActualizado: Cine): boolean
+agregarPelicula(nuevaPelicula: Pelicula): void
+obtenerPelicula(titulo: String): Pelicula
+eliminarPelicula(titulo: String): boolean
+modificarPelicula(tituloActual: String, modificacion: Pelicula): boolean
+buscarPeliculasPorActor(actorBuscado: String): List<Pelicula>
+buscarPeliculasPorClasificacion(clasificacion: String): List<Pelicula>
+programarFuncion(idCine: String, idSala: String, nuevaFuncion: Funcion): boolean
+eliminarFuncion(nombreCine: String, idSala: String, fecha: String, hrs: int): boolean
+modificarFuncion(nombreCine: String, idSala: String, fecha: String, hrs: int, nuevaFuncion: Funcion): boolean
+cancelarFuncion(idCine: String, idSala: String, funcionACancelar: Funcion): boolean
+registrarBoleto(boleto: Boleto): void
EstadoAsiento
DISPONIBLE
OCUPADO
RESERVADO
TipoSala
TRADICIONAL
TRES_D
VIP
IMAX
Usuario
-nombre: String
+Usuario(nombre: String)
+getNombre(): String
+setNombre(nombre: String): void
Administrador
-pinAcceso: String
+Administrador(nombre: String, pinAcceso: String)
+getPinAcceso(): String
+setPinAcceso(pinAcceso: String): void
+validarPin(pinIngresado: String): boolean
+registrarPelicula(nuevaPelicula: Pelicula, sistemaActivo: Sistema): void
+modificarPelicula(tituloActual: String, modificacion: Pelicula, sistemaActivo: Sistema): boolean
+eliminarPelicula(tituloEliminar: String, sistemaActivo: Sistema): boolean
+obtenerPelicula(titulo: String, sistemaActivo: Sistema): Pelicula
+programarFuncion(idCine: String, idSala: String, nuevaFuncion: Funcion, sistemaActivo: Sistema): boolean
+eliminarFuncion(idCine: String, idSala: String, funcionACancelar: Funcion, sistemaActivo: Sistema): boolean
+modificarFuncion(idCine: String, idSala: String, fActual: Funcion, fMod: Funcion, sistemaActivo: Sistema): boolean
Cine
-nombreCine: String
-salas: Map<String, Sala>
+Cine(nombreCine: String)
+getNombreCine(): String
+setNombreCine(nombre: String): void
+getSalas(): Map<String, Sala>
+setSalas(salas: Map<String, Sala>): void
+agregarSala(idSala: String, nuevaSala: Sala): void
+obtenerSala(idSala: String): Sala
+eliminarSala(idSala: String): boolean
+modificarSala(idActual: String, nuevoId: String, salaActualizada: Sala): boolean
+programarFuncion(idSala: String, nuevaFuncion: Funcion): boolean
+cancelarFuncion(idSala: String, funcionACancelar: Funcion): boolean
+modificarFuncion(idSala: String, actual: Funcion, modificada: Funcion): boolean
Cliente
+Cliente(nombre: String)
+comprarBoleto(funcion: Funcion, idSala: String, seleccion: List<Asiento>, metodoPago: MetodoDePago): Boleto
Exception
«exception»
EntidadNoEncontradaException
+EntidadNoEncontradaException(mensaje: String)
Sala
-idSala: String
-tipoSala: String
-precioBase: double
-filas: int
-asientosPorFila: int
-asientos: Map<String, Asiento>
-programacion: List<Funcion>
+Sala(idSala: String)
+Sala(idSala: String, filas: int, asientosFila: int, precioBase: double)
+getIdSala(): String
+getTipoSala(): String
+getPrecioBase(): double
+getTotalAsientos(): int
+getAsientos(): Map<String, Asiento>
+getProgramacion(): List<Funcion>
+agregarAsiento(asiento: Asiento): void
+obtenerAsiento(idAsiento: String): Asiento
+generarAsientos(filas: int, asientosPorFila: int, tipo: String, precio: double): void
+agregarFuncion(nuevaFuncion: Funcion): boolean
+programarFuncion(nuevaFuncion: Funcion): boolean
+cancelarFuncion(fecha: String, horarioHrs: int): boolean
+cancelarFuncion(funcionACancelar: Funcion): boolean
+modificarFuncion(actual: Funcion, modificada: Funcion): boolean
+mostrarAsientos(): void
«exception»
CredSistemaYaExistenteException
+CredSistemaYaExistenteException(mensaje: String)
Funcion
-fecha: String
-horarioHrs: int
-horarioMin: int
-peliculaProyectada: Pelicula
-sala: Sala
-asientosOcupados: List<Asiento>
-mapaAsientos: Map<String, Asiento>
+Funcion(fecha: String, hrs: int, min: int, peli: Pelicula)
+getFecha(): String
+getHorarioHrs(): int
+getHorarioMin(): int
+getPeliculaProyectada(): Pelicula
+getSala(): Sala
+setSala(sala: Sala): void
+inicializarAsientos(asientosSala: Map<String, Asiento>): void
+obtenerAsiento(identificador: String): Asiento
Boleto
-informacionPelicula: Pelicula
-informacionSala: String
-informacionFuncion: Funcion
-asientosAsignados: List<Asiento>
-totalPagado: double
-metodoPago: MetodoDePago
+Boleto(pelicula: Pelicula, sala: String, funcion: Funcion, asientos: List<Asiento>, pago: MetodoDePago)
+getInformacionPelicula(): Pelicula
+getInformacionSala(): String
+getInformacionFuncion(): Funcion
+getAsientosAsignados(): List<Asiento>
+getTotalPagado(): double
+setTotalPagado(total: double): void
+getMetodoPago(): MetodoDePago
+calcularTotal(): double
+imprimirBoleto(): void
Asiento
-identificador: String
-tipoAsiento: String
-estadoAsiento: String
-precio: double
+Asiento(identificador: String, tipoAsiento: String, precio: double)
+getIdentificador(): String
+getTipoAsiento(): String
+getEstadoAsiento(): String
+getPrecio(): double
+reservar(): boolean
+vender(): boolean
Pelicula
-titulo: String
-duracion: String
-clasificacion: String
-genero: String
-director: String
-actores: List<String>
-sinopsis: String
+Pelicula(titulo: String, duracion: String, clasif: String, genero: String, dir: String, actores: List<String>, sinopsis: String)
+getTitulo(): String
+getDuracion(): String
+getClasificacion(): String
+getGenero(): String
+getDirector(): String
+getActores(): List<String>
+getSinopsis(): String
«exception»
AsientoOcupadoException
+AsientoOcupadoException(mensaje: String)
MetodoDePago
+procesarPago(monto: double): boolean
TransferenciaElectronica
-cuentaBancaria: String
+TransferenciaElectronica(cuentaBancaria: String)
+getCuentaBancaria(): String
+procesarPago(monto: double): boolean
Paypal
-correo: String
+Paypal(correo: String)
+getCorreo(): String
+procesarPago(monto: double): boolean
TarjetaCredito
-numeroTarjeta: String
+TarjetaCredito(numeroTarjeta: String)
+getNumeroTarjeta(): String
+procesarPago(monto: double): boolean
gestiona (Map)
administra (Map)
contiene (Map)
registra (List)
catalogo (Map)
programacion (List)
proyecta
pelicula
mapa (Map)
butacas
pagado con
distribucion (Map<IdAsiento, Asiento>)
«throws»

Tecnologias Orientadas A Objetos

Avance 2 del Proyecto

Pablo De Jesús Salazar Rioú —-- Clave: 350287

Francisco Javier Silos Villegas —-- Clave 333127

Froylan Eloy Hernandez Castro

Fecha de Entrega: 13 de noviembre del 2026 del 2026

Cambios Agregados al Uml

Cambios echos en las clases:

En la clase Sistema:

- **Antes**
  - `cines : List<Cine>`
  - **catalogoPeliculas : List<Pelicula>**
- **Ahora**
  - `cines : Map<String, Cine>` (aqui la clave es el nombre del cine)
  - `catalogoPeliculas : Map<String, Pelicula>` (aqui la clave es el título de la película)

En la clase `Cine`:

- **Antes:** `salas : List<Sala>`
- **Ahora:** `salas : Map<String, Sala>` asociando ahora `idsala` con su objeto `Sala`).

En la clase `Sala`:

- **Antes:** `asientos : List<Asiento>`
- **Ahora:** `asientos : Map<String, Asiento>` (donde la clave es la coordenada de la butaca `"A1"`).

En la clase `Funcion`:

- **Antes:** `asientos : List<Asiento>`
- **Ahora:** `asientos : Map<String, Asiento>`
- **Método actualizado:** `inicializarAsientos(plantillaAsientos : Map<String, Asiento>) : void` cada función clona los asientos de la sala a su propio mapa para que no se empalmen en los horarios

ademas de implementaciones en el menu de la terminal

Implementacion de nuevas funciones en el menu

Se añadieron nuevas mini menus para lo que son las salas, peliculas, funciones

`menuAdministrado`: Panel con control de acceso por pin

`menuGestionarCines`: Permite dar de alta cines, listar los que ya existen y eliminar sucursales del mapa

`menuGestionarSalas`: nos permite elegir un cine específico y agregarle salas configurando filas, columnas y precio base.

`menuGestionarPeliculas` Permite registrar películas en el catálogo y darlas de baja.

`menuGestionarFunciones`: Permite añadir funciones al vinculando cine ➜ sala ➜película ➜ fecha y hora.

`menuCliente`: Permite consultar cartelera y hacer el flujo de compra seleccionando película, butaca y forma de pago.

Pruebas Unitarias (Junit 5) y Excepciones}

Metodos criticos

Método 1: `Asiento.vender()`

¿Por qué es crítico?

Por qué no podemos vender un mimo asiento dos veces por una simple razón, por el simple hecho de que en un mismo asiento no puede haber dos personas, ejemplo la persona A compra el asiento A1 y lo paga y llega una persona B queriendo comprar este mismo asiento si esta logra comprarlo al llegar a dicho asiento ya habría una persona sentado en el.Por esto este método debe garantizar que al compra un boleto este cambia a vendido y si alguien intenta comprar este otra vez se lance Asiento Ocupado Exception.

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class AsientoTest {

    // METODO CRITICO 1 Asiento.vender()

    // 1. Camino Feliz: Asiento disponible se vende exitosamente
    @Test
    public void testVender_CaminoFeliz() throws AsientoOcupadoException {
        Asiento asiento = new Asiento(identificador: "A1", tipoAsiento: "TRADICIONAL", precio: 80.0);
        boolean resultado = asiento.vender();
        assertTrue(resultado);
        assertEquals("VENDIDO", asiento.getEstadoAsiento());
    }

    // 2. Camino Triste: Intentar vender un asiento ya vendido lanza AsientoOcupadoException
    @Test
    public void testVender_CaminoTriste() throws AsientoOcupadoException {
        Asiento asiento = new Asiento(identificador: "A1", tipoAsiento: "TRADICIONAL", precio: 80.0);
        asiento.vender(); // Primera venta OK

        assertThrows(AsientoOcupadoException.class, () -> {
            asiento.vender(); // Segunda venta debe fallar
        });
    }
}
```

```java
    public boolean vender() throws AsientoOcupadoException {
        if (this.estadoAsiento.equalsIgnoreCase("VENDIDO")) {
            throw new AsientoOcupadoException("Error: El asiento " + this.identificador + " ya se encuentra vendido.");
        }
        this.estadoAsiento = "VENDIDO";
        return true;
    }
}
```

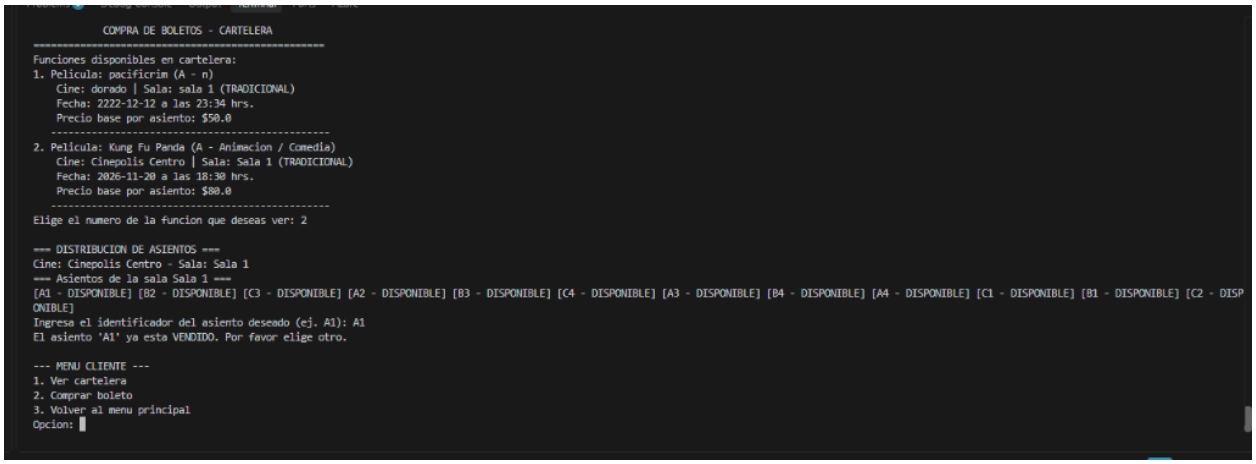
Ejemplo usando la terminal:

Primera compra hecha con exito

```
========================================
       ¡COMPRA REALIZADA CON EXITO!
========================================
   Boletos A Comprar
Pelicula: Kung Fu Panda
Clasificacion: A
Sala: Sala 1
Fecha: 2026-11-20
Horario: 18:30
Asientos: A1
Total Pagado: $80.0
```

Al intentar comprar el mismo te suelta un mensaje diciendo que el asiento que quieres ya ha sido vendido, por lo que no te deja comprarlo

```
Ingresa el identificador del asiento deseado (ej. A1): A1
El asiento 'A1' ya esta VENDIDO. Por favor elige otro.
```



hay uunas cosas a mejorar hay dice que el asiento A1 sigue disponible cunado lo acabo de comprar al comprarlo otra ves si nos marca el mensaje de que este asiento ya esta vendido pero no lo muetsra en la matriz,

Método 2: `Sala.agregarFuncion(nuevaFuncion)`:

¿Por qué es crítico?

Por que en un mismo horario no puede haber 2 películas proyectándose al mismo tiempo, así que si pasa esto se debería lanzar ConflictoHorarioException



```java
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import static org.junit.jupiter.api.Assertions.*;

public class SalaTest {

    // METODO CRITICO 2: Sala.agregarFuncion()

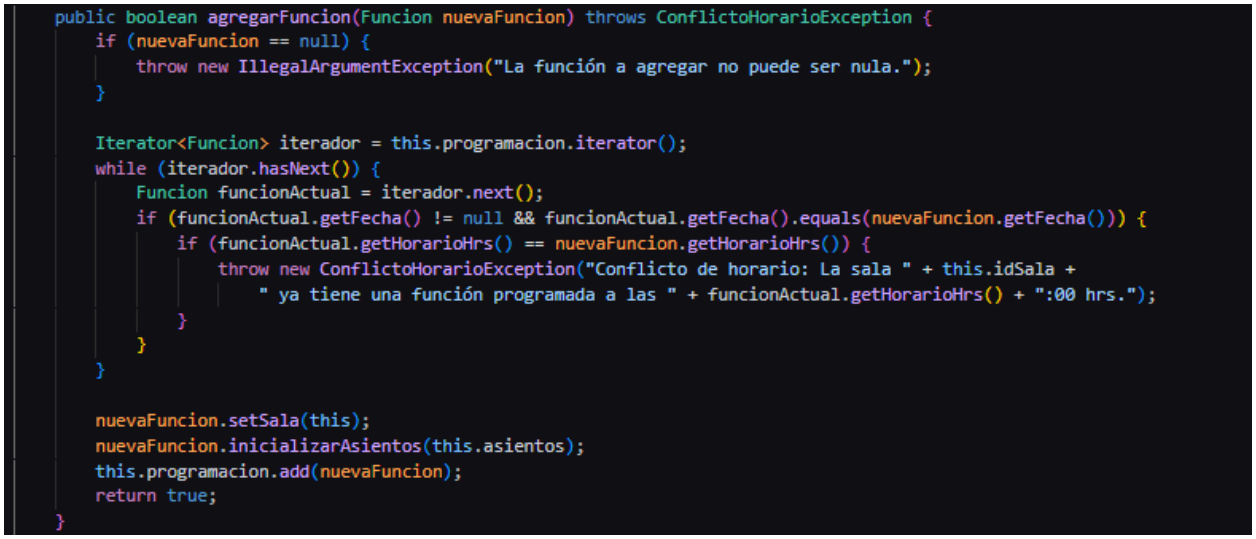
    // 1. Camino Feliz: Agregar funcion en horario libre
    @Test
    public void testAgregarFuncion_CaminoFeliz() throws ConflictoHorarioException {
        Sala sala = new Sala(idSala: "Sala 1", filas: 5, asientosPorFila: 5, precioBase: 80.0);
        Pelicula peli = new Pelicula(titulo: "Avatar", duracion: "2:40", clasificacion: "B", genero: "Accion", direct… "James Cameron", new ArrayList<>(), "Pandora");
        Funcion f1 = new Funcion(fecha: "2026-12-01", horarioHrs: 18, horarioMin: 0, peli);

        boolean agregada = sala.agregarFuncion(f1);
        assertTrue(agregada);
        assertEquals(1, sala.getProgramacion().size());
    }

    // 2. Camino Triste: Agregar dos funciones con el mismo horario lanza ConflictoHorarioException
    @Test
    public void testAgregarFuncion_CaminoTriste() throws ConflictoHorarioException {
        Sala sala = new Sala(idSala: "Sala 1", filas: 5, asientosPorFila: 5, precioBase: 80.0);
        Pelicula peli = new Pelicula(titulo: "Avatar", duracion: "2:40", clasificacion: "B", genero: "Accion", direct… "James Cameron", new ArrayList<>(), "Pandora");
        Funcion f1 = new Funcion(fecha: "2026-12-01", horarioHrs: 18, horarioMin: 0, peli);
        Funcion f2 = new Funcion(fecha: "2026-12-01", horarioHrs: 18, horarioMin: 0, peli); // Mismo horario

        sala.agregarFuncion(f1);

        assertThrows(ConflictoHorarioException.class, () -> {
            sala.agregarFuncion(f2);
        });
    }
}
```



```java
public boolean agregarFuncion(Funcion nuevaFuncion) throws ConflictoHorarioException {
    if (nuevaFuncion == null) {
        throw new IllegalArgumentException("La función a agregar no puede ser nula.");
    }

    Iterator<Funcion> iterador = this.programacion.iterator();
    while (iterador.hasNext()) {
        Funcion funcionActual = iterador.next();
        if (funcionActual.getFecha() != null && funcionActual.getFecha().equals(nuevaFuncion.getFecha())) {
            if (funcionActual.getHorarioHrs() == nuevaFuncion.getHorarioHrs()) {
                throw new ConflictoHorarioException("Conflicto de horario: La sala " + this.idSala +
                    " ya tiene una función programada a las " + funcionActual.getHorarioHrs() + ":00 hrs.");
            }
        }
    }

    nuevaFuncion.setSala(this);
    nuevaFuncion.inicializarAsientos(this.asientos);
    this.programacion.add(nuevaFuncion);
    return true;
}
```

Ejemplo en consola:

Primera función programada

```
--- PASO 1: SELECCIONA EL CINE ---
1. dorado (Salas: 1)
2. Cinepolis Centro (Salas: 1)
Numero de cine (o nombre): 1

--- PASO 2: SELECCIONA LA SALA EN DORADO ---
1. sala 1 (TRADICIONAL - 25 asientos - $50.0)
Numero de sala (o ID): 1

--- PASO 3: SELECCIONA LA PELICULA A PROYECTAR ---
1. Iron Man 2 (B - Accion [2:04])
2. Kung Fu Panda (A - Animacion / Comedia [1:32])
3. pacificrim (A - n [1:21])
Numero de pelicula (o titulo): 1
Fecha de proyeccion (AAAA-MM-DD): 2026-11-12
Hora de inicio (0-23): 14
Minutos de inicio (0-59): 30
¡Funcion programada exitosamente!
  ? Cine: dorado
  ? Sala: sala 1
  ? Pelicula: Iron Man 2
  ? Horario: 2026-11-12 a las 14:30 hrs.
```

ahora, programamos la misma función por segunda vez para ver que pasa:

```
1. Programar nueva Funcion
2. Ver funciones programadas por Cine
3. Volver
Opcion: 1

--- PASO 1: SELECCIONA EL CINE ---
1. dorado (Salas: 1)
2. Cinepolis Centro (Salas: 1)
Numero de cine (o nombre): 1

--- PASO 2: SELECCIONA LA SALA EN DORADO ---
1. sala 1 (TRADICIONAL - 25 asientos - $50.0)
Numero de sala (o ID): 1

--- PASO 3: SELECCIONA LA PELICULA A PROYECTAR ---
1. Iron Man 2 (B - Accion [2:04])
2. Kung Fu Panda (A - Animacion / Comedia [1:32])
3. pacificrim (A - n [1:21])
Numero de pelicula (o titulo): 1
Fecha de proyeccion (AAAA-MM-DD): 2026-11-12
Hora de inicio (0-23): 14
Minutos de inicio (0-59): 30
Error de horario: Conflicto de horario: La sala sala 1 ya tiene una función programada a las 14:00 hrs.
```

Método 3:`Sistema.obtener Pelicula(titulo)`:

¿Por qué es crítico?

Por que al momento de agregar una nueva película al catálogo etsa debe de existir por lo que el admin debe de escribir correctamente el nombre de la película y la pantalla no debería de quedarse en blanco si no que debería de lanzar un mensaje de EntidadNoEncontradaException

```java
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import static org.junit.jupiter.api.Assertions.*;

public class SistemaTest {

    // METODO CRITICO 3: Sistema.obtenerPelicula()

    @Test
    public void testObtenerPelicula_CaminoFeliz() throws EntidadNoEncontradaException {
        Sistema sistema = new Sistema();
        Pelicula p = new Pelicula(titulo: "Batman", duracion: "2:10", clasificacion: "B", genero: "Accion", direct… "Matt Reeves", new ArrayList<>(), "Gotham");
        sistema.agregarPelicula(p);

        Pelicula resultado = sistema.obtenerPelicula(titulo: "Batman");
        assertNotNull(resultado);
        assertEquals("Batman", resultado.getTitulo());
    }

    @Test
    public void testObtenerPelicula_CaminoTriste() {
        Sistema sistema = new Sistema();

        assertThrows(EntidadNoEncontradaException.class, () -> {
            sistema.obtenerPelicula(titulo: "PeliculaFantasma");
        });
    }
```

```java
public Pelicula obtenerPelicula(String titulo) throws EntidadNoEncontradaException {
    if (titulo == null || titulo.trim().isEmpty()) {
        throw new IllegalArgumentException("El título de la película no puede estar vacío.");
    }
    if (this.catalogoPeliculas == null) {
        throw new EntidadNoEncontradaException(mensaje: "El catálogo de películas está vacío.");
    }
    String busqueda = titulo.trim().toLowerCase();

    for (Pelicula p : this.catalogoPeliculas.values()) {
        if (p.getTitulo().equalsIgnoreCase(busqueda)) {
            return p;
        }
    }
    for (Pelicula p : this.catalogoPeliculas.values()) {
        if (p.getTitulo().toLowerCase().contains(busqueda)) {
            return p;
        }
    }
    throw new EntidadNoEncontradaException("La película '" + titulo + "' no fue encontrada en el catálogo.");
}
```

Ejemplo en consola:

Agregada correctamente la función

```
--- PASO 3: SELECCIONA LA PELICULA A PROYECTAR ---
1. Iron Man 2 (B - Accion [2:04])
2. Kung Fu Panda (A - Animacion / Comedia [1:32])
3. batman23 (gt - gt [gt])
4. pacificrim (A - n [1:21])
Numero de pelicula (o titulo): iron man 3
Error: La película 'iron man 3' no fue encontrada en el catálogo.

--- GESTION DE FUNCIONES ---
```

agregandola usando un nombre incorrecto

```
--- PASO 3: SELECCIONA LA PELICULA A PROYECTAR ---
1. Iron Man 2 (B - Accion [2:04])
2. Kung Fu Panda (A - Animacion / Comedia [1:32])
3. batman23 (gt - gt [gt])
4. pacificrim (A - n [1:21])
Numero de pelicula (o titulo): Iron man 2
Fecha de proyeccion (AAAA-MM-DD): 2026-11-23
Hora de inicio (0-23): 23
Minutos de inicio (0-59): 55
¡Funcion programada exitosamente!
  ? Cine: dorado
  ? Sala: sala 1
  ? Pelicula: Iron Man 2
  ? Horario: 2026-11-23 a las 23:55 hrs.
```

.

Método 4 :`Boleto.calcularTotal()`:

¿Por qué es crítico?

Por que si al calcular el total de los boletos comprados se haya calculado por centavos de más o de menos esto representaría un gran problema tanto para el cine o para el usuario y para que esto no pase se debe sumar con precisión y verificar que las butacas estén disponibles y no tengan precios negativos.

```java
    // METODO CRITICO 4: Boleto.calcularTotal()

    @Test
    public void testCalcularTotal_CaminoFeliz() throws AsientoOcupadoException {
        Pelicula p = new Pelicula(titulo: "Toy Story", duracion: "1:30", clasificacion: "A", genero: "Animacion", direct… "Director", new ArrayList<>(), "Sinopsis");
        Funcion f = new Funcion(fecha: "2026-11-20", horarioHrs: 15, horarioMin: 0, p);

        Asiento a1 = new Asiento(identificador: "A1", tipoAsiento: "TRADICIONAL", precio: 80.0);
        Asiento a2 = new Asiento(identificador: "A2", tipoAsiento: "TRADICIONAL", precio: 80.0);
        a1.vender();
        a2.vender();

        List<Asiento> lista = new ArrayList<>();
        lista.add(a1);
        lista.add(a2);

        Boleto boleto = new Boleto(p, informacionSala: "Sala 1", f, lista, new Paypal(correo: "test@cine.com"));
        double total = boleto.calcularTotal();

        assertEquals(160.0, total, 0.01);
    }

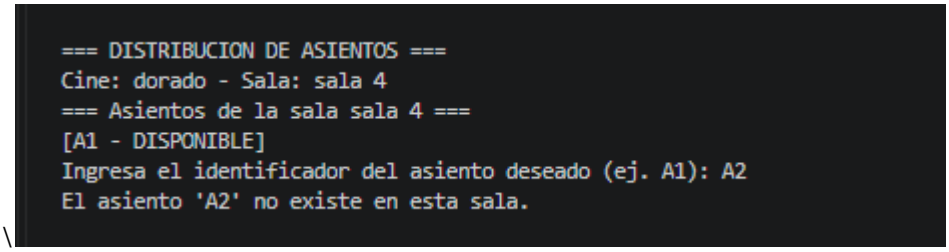
    @Test
    public void testCalcularTotal_CaminoTriste() {
        Pelicula p = new Pelicula(titulo: "Toy Story", duracion: "1:30", clasificacion: "A", genero: "Animacion", direct… "Director", new ArrayList<>(), "Sinopsis");
        Funcion f = new Funcion(fecha: "2026-11-20", horarioHrs: 15, horarioMin: 0, p);

        // Sin asientos asignados -> el constructor o calcularTotal() debe lanzar IllegalStateException
        assertThrows(IllegalStateException.class, () -> {
            new Boleto(p, informacionSala: "Sala 1", f, new ArrayList<>(), new Paypal(correo: "test@cine.com"));
        });
    }
}
```

Se compró el primer boleto de forma correcta

```
   Boletos A Comprar
Pelicula: Iron Man 2
Clasificacion: B
Sala: sala 4
Fecha: 2026-11-14
Horario: 23:45
Asientos: A1
Total Pagado: $50.0
```

Se intento comprar el asiento de la butaca A2 pero como no existe salta la alerta de que el asiento A2 no existe en esta sala



\

Método 5 :`TarjetaCredito.procesarPago(monto)`:

¿Por qué es crítico?

Aquí es donde valida las reglas de los bancos o de las diversas formas de pago osea que debe asegurarse que que el monto sea mayor a 0 o que el número de la tarjeta cumpla con la estructura requerida que serían de 16 dígitos en total, en caso que falle debe lanzar PagoRechazadoException

```java
public class TarjetaCreditoTest {

    // METODO CRITICO 5: TarjetaCredito.procesarPago()

    @Test
    public void testProcesarPago_CaminoFeliz() {
        TarjetaCredito tc = new TarjetaCredito(numeroTarjeta: "1234567890123456");
        boolean resultado = tc.procesarPago(monto: 150.0);
        assertTrue(resultado);
    }

    @Test
    public void testProcesarPago_CaminoTriste() {
        TarjetaCredito tc = new TarjetaCredito(numeroTarjeta: "1234567890123456");

        // Monto negativo -> IllegalArgumentException
        assertThrows(IllegalArgumentException.class, () -> {
            tc.procesarPago(-50.0);
        });
    }
}
```

```java
@Override
public boolean procesarPago(double monto) {
    if (monto <= 0) {
        throw new IllegalArgumentException("El monto a pagar debe ser mayor a 0 pesos. Monto recibido: " + monto);
    }
    if (this.numeroTarjeta == null || this.numeroTarjeta.trim().length() < 16) {
        throw new IllegalArgumentException("Número de tarjeta inválido para procesar la transacción.");
    }
    System.out.println("Pago de $" + monto + " aprobado con Tarjeta terminación " + this.numeroTarjeta.substring(this.numeroTarjeta.length() - 4));
    return true;
}
}
```

Se compró un boleto con el número total de dígitos válidos para una tarjeta de crédito que son de 16 dígitos :

```
Selecciona tu metodo de pago:
1. Tarjeta de Credito
2. Paypal
3. Transferencia Electronica
Opcion: 1
Numero de tarjeta (16 digitos): 1234123412341234
Pago de $80.0 aprobado con Tarjeta terminación 1234
Compra exitosa para el cliente pj

=======================================
       ¡COMPRA REALIZADA CON EXITO!
=======================================
   Boletos A Comprar
Pelicula: Kung Fu Panda
Clasificacion: A
Sala: Sala 1
Fecha: 2026-11-20
Horario: 18:30
Asientos: A2
Total Pagado: $80.0
```

Aquí intentamos comprar un boleto pero con un número de tarjeta erróneo:

nos suelta la alerta de número de tarjeta erróneo

```
=== Asientos de la sala Sala 1 ===
[A1 - DISPONIBLE] [B2 - DISPONIBLE] [C3 - DISPONIBLE] [A2 - DISPONIBLE] [B3 - DIS
ONIBLE]
Ingresa el identificador del asiento deseado (ej. A1): C3

Detalle: Asiento C3 (TRADICIONAL) - Precio: $80.0
Selecciona tu metodo de pago:
1. Tarjeta de Credito
2. Paypal
3. Transferencia Electronica
Opcion: 1
Numero de tarjeta (16 digitos): 1

[ERROR DE PAGO] Número de tarjeta inválido para procesar la transacción.
```

Método 6 :`Sistema.eliminarCine(nombreCine)`:

¿Por qué es crítico?

Por que al eliminar un cine se estaría eliminando en cascada las salas de los cines sus funciones sus asientos así que se deberá verificar con exactitud el nombre del cien a eliminar y en caso de que no exista lanzar EntidadNoEncontradaException para no desequilibrar el orden de los demás cines.

```java
    @Test
    public void testEliminarCine_CaminoFeliz() throws EntidadNoEncontradaException {
        Sistema sistema = new Sistema();
        sistema.agregaCine(new Cine(nombreCine: "Cinepolis Tangamanga"));

        boolean eliminado = sistema.eliminarCine(nombreCine: "Cinepolis Tangamanga");
        assertTrue(eliminado);
        assertEquals(0, sistema.getCines().size());
    }

    @Test
    public void testEliminarCine_CaminoTriste() {
        Sistema sistema = new Sistema();

        assertThrows(EntidadNoEncontradaException.class, () -> {
            sistema.eliminarCine(nombreCine: "CineQueNoExiste");
        });
    }
}
```

```java
public Cine obtenerCine(String nombreCine) throws EntidadNoEncontradaException {
    if (nombreCine == null || nombreCine.trim().isEmpty()) {
        throw new IllegalArgumentException("El nombre del cine no puede estar vacío.");
    }
    if (this.cines == null) {
        throw new EntidadNoEncontradaException(mensaje: "El catálogo de cines está vacío.");
    }
    String busqueda = nombreCine.trim().toLowerCase();

    // 1. Coincidencia exacta
    for (Cine c : this.cines.values()) {
        if (c.getNombreCine().equalsIgnoreCase(busqueda)) {
            return c;
        }
    }
    // 2. Coincidencia parcial
    for (Cine c : this.cines.values()) {
        if (c.getNombreCine().toLowerCase().contains(busqueda)) {
            return c;
        }
    }
    throw new EntidadNoEncontradaException("El cine '" + nombreCine + "' no existe en el sistema.");
}
```

Eliminamos un cine que existe correctamente:

```
--- GESTION DE CINES ---
1. Registrar nuevo Cine
2. Listar Cines registrados
3. Eliminar Cine
4. Volver
Opcion: 3
Nombre del cine a eliminar: Cinepolis Centro
Cine eliminado correctamente.
```

si eliminamos un cine que no existe en el sistema nos suelta un mensaje de error:

Asi evitamos que no eliminemos un cine erróneo por error

```
--- GESTION DE CINES ---
1. Registrar nuevo Cine
2. Listar Cines registrados
3. Eliminar Cine
4. Volver
Opcion: 3
Nombre del cine a eliminar: cine dorado
Error: El cine 'cine dorado' no existe en el sistema.
```

Método 7:`Cine.obtenerSala(idSala)`:

¿Por qué es crítico?

Porque al programar una función , el sistema busca la sala dentro del mapa de los cines , si no existe la búsqueda debe de fallar de inmediato evitando que se vendan boletos en una sala que ni siquiera existe.

```java
public class CineTest {

    // METODO CRITICO 7: Cine.obtenerSala()

    @Test
    public void testObtenerSala_CaminoFeliz() throws EntidadNoEncontradaException {
        Cine cine = new Cine(nombreCine: "Cinepolis Test");
        Sala sala = new Sala(idSala: "Sala 1", filas: 5, asientosPorFila: 5, precioBase: 80.0);
        cine.agregaSala(idSala: "Sala 1", sala);

        Sala encontrada = cine.obtenerSala(idSala: "Sala 1");
        assertNotNull(encontrada);
        assertEquals("Sala 1", encontrada.getIdSala());
    }

    @Test
    public void testObtenerSala_CaminoTriste() {
        Cine cine = new Cine(nombreCine: "Cinepolis Test");

        // Sala que no existe -> EntidadNoEncontradaException
        assertThrows(EntidadNoEncontradaException.class, () -> {
            cine.obtenerSala(idSala: "Sala 99");
        });
    }
}
```

```java
public Sala obtenerSala(String idSala) throws EntidadNoEncontradaException {
    if (this.salas == null || !this.salas.containsKey(idSala)) {
        throw new EntidadNoEncontradaException("La sala '" + idSala + "' no existe en el cine " + this.nombreCine);
    }
    return this.salas.get(idSala);
}
```

Aqui programamos una función en una sala que existe:

```
--- PASO 1: SELECCIONA EL CINE ---
1. Cinepolis Centro (Salas: 1)
Numero de cine (o nombre): 1

--- PASO 2: SELECCIONA LA SALA EN CINEPOLIS CENTRO ---
1. Sala 1 (TRADICIONAL - 12 asientos - $80.0)
Numero de sala (o ID): Sala 1

--- PASO 3: SELECCIONA LA PELICULA A PROYECTAR ---
1. Iron Man 2 (B - Accion [2:04])
2. Kung Fu Panda (A - Animacion / Comedia [1:32])
Numero de pelicula (o titulo): 1
Fecha de proyeccion (AAAA-MM-DD): 2026-12-12
Hora de inicio (0-23): 23
Minutos de inicio (0-59): 45
¡Funcion programada exitosamente!
  ? Cine: Cinepolis Centro
  ? Sala: Sala 1
  ? Pelicula: Iron Man 2
  ? Horario: 2026-12-12 a las 23:45 hrs.
```

y aqui programamos una función en una sala que no existe

nos devuelve error al instante evitando que se programen funciones en salas inexistentes

```
--- PASO 1: SELECCIONA EL CINE ---
1. Cinepolis Centro (Salas: 1)
Numero de cine (o nombre): sala 99
Error: El cine 'sala 99' no existe en el sistema.
```

Método 8 :`Administrador.validarPin(pin)`

¿Por qué es crítico?

El administrador es el que tiene permisos para toda la estructura del cine salas funciones boletos etc etc se debe verificar que solo el administrador pueda entrar así evitando que cualquier otra persona que no sea el administrador pueda entrar a modificar la estructura de los cines.

```
    // METODO CRITICO 8: Administrador.validarPin()

    @Test
    public void testValidarPin_CaminoFeliz() {
        Administrador admin = new Administrador(nombre: "Carlos", pinAcceso: "1234");
        assertTrue(admin.validarPin(pinIngresado: "1234"));
        assertFalse(admin.validarPin(pinIngresado: "9999"));
    }

    @Test
    public void testValidarPin_CaminoTriste() {
        Administrador admin = new Administrador(nombre: "Carlos", pinAcceso: "1234");

        // PIN nulo -> IllegalArgumentException
        assertThrows(IllegalArgumentException.class, () -> {
            admin.validarPin(pinIngresado: null);
        });
    }
}
```

```
public boolean validarPin(String pinIngresado) {
    if (pinIngresado == null || pinIngresado.trim().isEmpty()) {
        throw new IllegalArgumentException("El PIN ingresado no puede ser nulo ni vacío.");
    }
    return this.pinAcceso != null && this.pinAcceso.equals(pinIngresado.trim());
}
```

Aqui entramos con nuestro usuario y pin ya previamente ingresados

```
--- REGISTRO DE NUEVO ADMINISTRADOR ---
Ingresa tu nombre de usuario: pj
Crea tu PIN secreto de acceso (minimo 4 digitos): 1111
¡Administrador 'pj' registrado con exito! Ya puedes iniciar sesion con tu PIN.

================================================
```

```
--- ACCESO DE ADMINISTRADOR ---
Nombre de usuario Administrador: pj
PIN secreto: 1111

¡Bienvenido al panel, Administrador pj!

========================================
```

esto pasa si ingresamos con un usuario o pin invalido

```
--- ACCESO DE ADMINISTRADOR ---
Nombre de usuario Administrador: kj
PIN secreto: 3221
Error: Credenciales incorrectas (Usuario inexistente o PIN invalido).

================================================
```